C# for Java Programmers

1. Introduction

If you already know Java, learning C# is relatively straightforward. Both languages use familiar object-oriented concepts and have similar syntax for variables, conditions, loops, classes, methods, and interfaces.

However, C# is not simply "Java with different syntax." It runs in the .NET ecosystem and provides several language features that Java programmers need to recognize.

This section focuses on those differences and shows how familiar Java concepts translate into C#.

Learning Objectives

By the end of this section, you should be able to:

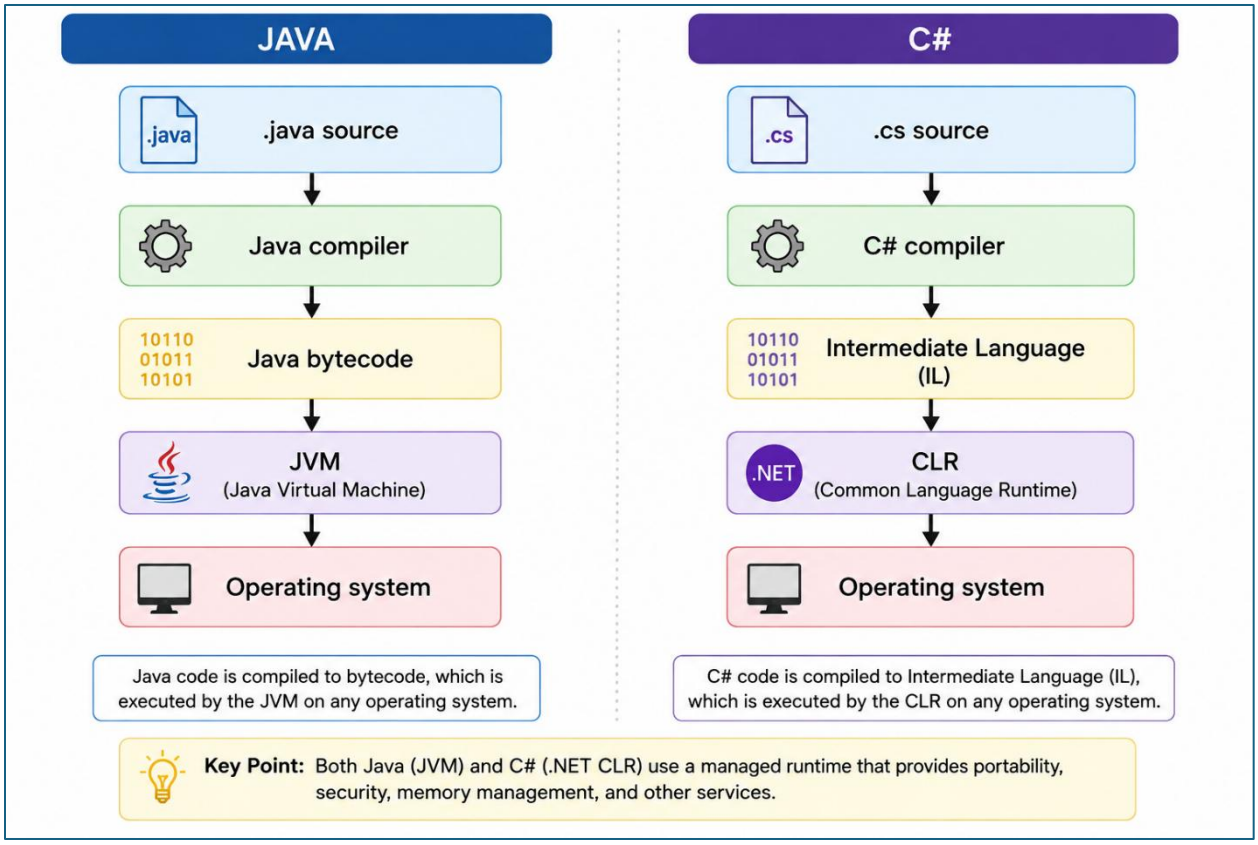
- identify important differences between Java and C#;
- recognize common C# keywords, operators, and data types;
- create a basic C# program and class;
- translate familiar Java syntax into C#;
- understand how C# properties differ from traditional Java getter/setter methods;
- recognize basic C# interface syntax;
- understand the role of C# structs and enums.

---

2. Java and C#: Similar Goals, Different Platforms

Both Java and C# are strongly typed, managed languages commonly used to build object-oriented applications. They also provide automatic memory management and large standard libraries.

Conceptual Comparison



Important Language Differences

The following table highlights several distinctions:

Area	Java	C#
Runtime	Java runtime/JVM ecosystem	CLR/.NET ecosystem
Programming model	Primarily class-based OOP	OOP with additional functional and component-oriented features
Operator overloading	Very limited/no user-defined general operator overloading	Supports user-defined operator overloading
Pointers	No conventional pointer support	Available in `unsafe` contexts
Arrays	Objects in the Java type system	Based on the .NET `System.Array` type

These are conceptual differences; most everyday C# programming still feels familiar to an experienced Java programmer.

Note: Pointer programming and `unsafe` C# are specialized subjects. At this stage, students only need to recognize that the capability exists.

---

3. Why Java Programmers Usually Adapt Quickly

Java and C# share many fundamental programming constructs. Both support reusable code, multithreaded programming, automatic memory management, and extensive libraries. The original slides also emphasize C# features such as lambda expressions, generics, LINQ, extension methods, properties, and garbage collection.

For this lesson, the most useful approach is therefore:

Do not relearn programming. Learn the C# equivalents of concepts you already know.

For example:

Java

```
int age = 20;

if (age >= 18) {
    System.out.println("Adult");
}
```

C#

```
int age = 20;

if (age >= 18)
{
    Console.WriteLine("Adult");
}
```

The algorithm is identical. Only some syntax and library names change.

---

4. C# Keywords

Like Java, C# reserves certain words for language constructs.

Examples include:

```
class
public
private
protected
static
```

```
void
int
double
bool
if
else
for
while
return
new
this
```
Many are immediately recognizable to Java programmers.

C# also contains keywords associated with C#-specific language features, such as:

```
struct
delegate
event
operator
implicit
explicit
```
Important Rule

A keyword normally cannot be used as an identifier.

For example:

```
int class = 5;    // Invalid
```

Tip: Do not try to memorize the entire keyword table. Focus first on recognizing the keywords that appear in actual programs.

---

5. Operators

Most operators familiar from Java behave similarly in C#.

5.1 Arithmetic Operators

```
+    -    *    /    %
```

Example:

```
int x = 10;
int y = 3;

Console.WriteLine(x + y);  // 13
```

```
Console.WriteLine(x % y);  // 1
```

5.2 Comparison Operators

```
==    !=    <    >    <=    >=
```

5.3 Logical Operators

&& || !

Example:

```
int age = 21;
bool registered = true;

if (age >= 18 && registered)
{
    Console.WriteLine("Access granted");
}
```

5.4 Assignment Operators

```
=

+=

-=

*=

/=

%=
```

A Notable C# Difference: Operator Overloading

The source specifically identifies operator overloading as an important difference between the languages.

C# allows classes and structs to define the behavior of certain operators for their own types.

Note: At this point, students only need to understand what operator overloading means. Implementing custom overloaded operators can be addressed when appropriate later in the course.

---

6. Common Data Types

Many primitive-looking types in C# closely correspond to Java types.

Java	C#	Example
boolean	bool	bool active = true;
byte	byte	byte x = 10;
short	short	short x = 100;
int	int	int age = 25;
long	long	long count = 5000L;
float	float	float rate = 2.5f;
double	double	double price = 19.99;
char	char	char grade = 'A';
String	string	string name = "Sara";

C# also provides types that do not have the same direct Java primitive equivalent, including:

```
decimal
uint
ulong
ushort
```

The slides additionally map familiar Java wrapper classes to C#/.NET types—for example, Java Integer to C# Int32 and Java Boolean to C# Boolean.

Example

```
int number = 10;
double average = 82.5;
bool passed = true;
char grade = 'A';
string student = "Alex";
```

Pitfall: Java uses `boolean`; C# uses `bool`. Java programmers frequently type `boolean` by habit when first switching to C#.

---

7. Your First C# Program

Consider a minimal console program:

```
using System;

class Program
{
```

```
    static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
    }
}
```

Let's examine it line by line.

using System;

```
using System;
```

This brings the `System` namespace into scope.

class Program

```
class Program
```

As in Java, executable code can be organized inside classes.

static void Main(...)

```
static void Main(string[] args)
```

Main is the application's entry point.

Console.WriteLine(...)

```
Console.WriteLine("Hello World!");
```

This is the familiar C# equivalent of:

```
System.out.println("Hello World!");
```

---

8. Lab: Translate a Small Java Program to C#

Let's use Java knowledge to construct a C# program.

Step 1 — Start with the Java Version

```
public class Student {
    private String name;
    private int grade;

    public Student(String name, int grade) {
        this.name = name;
        this.grade = grade;
    }

    public void display() {
```

```
        System.out.println(name + ": " + grade);
    }
}
```

Step 2 — Change Java Types to C# Types

String becomes string:

```
private string name;
private int grade;
```

Step 3 — Translate the Constructor

The structure remains almost identical:

```
public Student(string name, int grade)
{
    this.name = name;
    this.grade = grade;
}
```

Step 4 — Translate Console Output

Replace:

```
System.out.println(...)
```

with:

```
Console.WriteLine(...)
```

Step 5 — Complete the C# Class

```
using System;

public class Student
{
    private string name;
    private int grade;

    public Student(string name, int grade)
    {
        this.name = name;
        this.grade = grade;
    }

    public void Display()
    {
        Console.WriteLine(name + ": " + grade);
    }
}
```

The important observation is that the **object-oriented design did not change**. We mainly translated language and library syntax.

---

9. Properties: An Important C# Feature

Properties are one of the most important differences for Java programmers.

Traditional Java Approach

```
private String name;

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}
```

The caller writes:

```
student.setName("Sara");
System.out.println(student.getName());
```

C# Property

C# can expose controlled access through a property:

```
private string name;

public string Name
{
    get
    {
        return name;
    }

    set
    {
        name = value;
    }
}
```

The caller can then write:

```
student.Name = "Sara";
Console.WriteLine(student.Name);
```

Notice that the code **looks like direct field access**, but get and set provide controlled access.

Key Concept: A C# property is not simply a public field. It provides accessor logic through get and set.

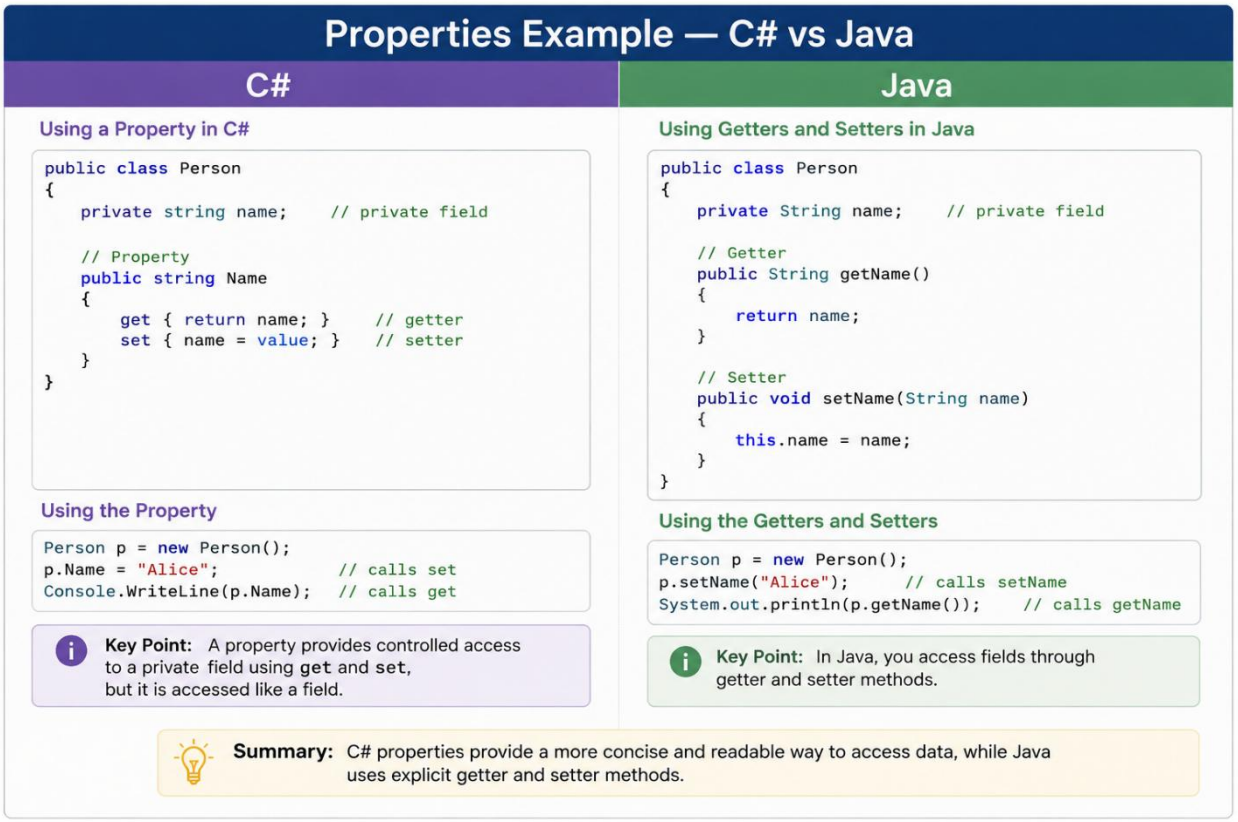
Practical Example

```
public class Student
{
    private int grade;

    public int Grade
    {
        get
        {
            return grade;
        }

        set
        {
            if (value >= 0 && value <= 100)
            {
                grade = value;
            }
        }
    }
}
```

Here, the setter validates the value before changing the field.



---

10. Interfaces: Java vs. C#

Interfaces exist in both languages and serve the same general purpose: defining a contract that implementing classes must follow.

A simple C# interface might look like:

```
interface IDataCalculator
{
    double Calculate();
}
```

A class implements it as follows:

```
class DataCalculator : IDataCalculator
{
    public double Calculate()
    {
        return 10.0;
    }
}
```

One immediately visible difference for Java programmers is the syntax used after the class name.

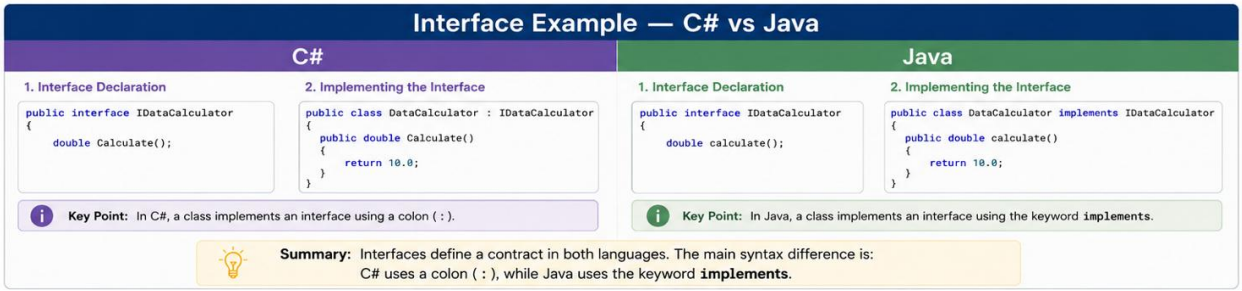
Java

```
class DataCalculator implements IDataCalculator
```

C#

```
class DataCalculator : IDataCalculator
```

Visual suggestion: Place the supplied **Interface Example — C# vs Java** comparison screenshot immediately after this example.



---

11. Structs and Enums

The following section includes a C# example involving struct and enum. These are useful C# constructs to recognize when transitioning from Java.

Enum

An enum defines a fixed collection of named constants.

```
enum Month
{
    January,
    February,
    March,
    April,
    May
}
```

Instead of representing a month with an arbitrary integer:

```
int month = 3;
```

we can represent it using a meaningful value:

```
Month month = Month.March;
```

This improves readability and reduces the use of unexplained numeric values.

Struct

C# also supports struct declarations.

```
Basic syntax:
struct Date
{
    public int Year;
    public Month Month;
    public int Day;
}
```
Example:

```
Date today;

today.Year = 2026;
today.Month = Month.September;
today.Day = 7;
```
Visual suggestion: Insert the supplied **C# Struct and Enum Example** screenshot here, after students have first seen the simplified versions above. The screenshot is much easier to understand once the two constructs have been introduced separately.

